Week 7 — Error Handling & Debugging (Spyder-friendly)

Outcome: you're less scared of errors and you know how to inspect and fix them like a calm code surgeon 🥼🪄

What we'll cover:

- Common Python error types (and what they *really* mean)
- Reading a traceback without panicking
- `try` / `except` / `else` / `finally`
- Raising your own errors (`raise`) and making custom exceptions
- Debugging workflow in **Spyder** (breakpoints, Variable Explorer, the Debugger pane)

1) Common error types (with tiny demos)

Read the error message **from bottom to top**. The last line is the exception type and message.

In [1]:
```
# Uncomment lines one at a time to see the errors.
# 1) NameError — using a variable that doesn't exist
print(not_defined_yet)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[1], line 3
      1 # Uncomment lines one at a time to see the errors.
      2 # 1) NameError — using a variable that doesn't exist
----> 3 print(not_defined_yet)

NameError: name 'not_defined_yet' is not defined
```

In [2]:
```
 # 2) TypeError — using a value of the wrong type
print('3' + 4)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[2], line 2
      1  # 2) TypeError — using a value of the wrong type
----> 2 print(     + 4)

TypeError: can only concatenate str (not "int") to str
```

In [3]:
```
# 3) ValueError — type is OK, but the *value* is wrong
# `int(3)` will return 3;
int("three")
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[3], line 3
      1 # 3) ValueError — type is OK, but the *value* is wrong
      2 # `int(3)` will return 3;
----> 3 int(        )

ValueError: invalid literal for int() with base 10: 'three'
```

In [4]:
```
# 4) IndexError — index outside the range
nums = [10, 20, 30]
nums[5]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[4], line 3
      1 # 4) IndexError — index outside the range
      2 nums = [10, 20, 30]
----> 3 nums[5]

IndexError: list index out of range
```

In [5]:
```
# 5) KeyError — missing key in a dict
d = {"a": 1}
d["b"]
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[5], line 3
      1 # 5) KeyError — missing key in a dict
      2 d = {"a": 1}
----> 3 d[   ]

KeyError: 'b'
```

In [6]:
```python
# 6) ZeroDivisionError — division by zero
1/0
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[6], line 2
      1 # 6) ZeroDivisionError — division by zero
----> 2 1/0

ZeroDivisionError: division by zero
```

2) How to read a traceback (without fear)

A traceback shows the **call stack**: which functions called which, ending where the error happened.

- Start at the bottom: `ExceptionType: message`
- Then move upwards to find the line **you** wrote that triggered it.

In [7]:
```python
def inner():
    return int("not-a-number")

def outer():
    return inner()

# Try running outer() and read the traceback from bottom to top
outer()
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[7], line 8
      5     return inner()
      7 # Try running outer() and read the traceback from bottom to top
----> 8 outer()

Cell In[7], line 5, in outer()
      4 def outer():
----> 5     return inner()

Cell In[7], line 2, in inner()
      1 def inner():
----> 2     return int(               )

ValueError: invalid literal for int() with base 10: 'not-a-number'
```

So `ExceptionType: message` in this case is `ValueError: invalid literal for int() with base 10: 'not-a-number'`. This means that unexpected *string* is provided to the function: `int()`.

We move up and read which line is causing this `ValueError`. Indicated by the arrow, it's `return int("not-a-number")`.

This is what we know:

```
>>> int("10")
10

>>> int("-3")
-3

>>> int("7.3")
ValueError: invalid literal for int() with base 10: 7.3
```

So you can pass a string of an integer (ex. 10 or -3), but anything else will be recognized as `invalid literal`.

3) `try` / `except` / `else` / `finally`

`try` / `except` lets your code *keep* running even when something goes wrong.

- `try`: code that *might* fail
- `except`: handle specific failures
- `else`: runs **only if no exception** happened

- `finally` : runs **no matter what** (cleanup, closing files, etc.)

The most simple structure has `try` and `except` only. Typically, your *main* lines that you want to run go under `try` . After `except` , you typically **print error messages**, **raise errors**, or provide an **alternative methods**.

ex.

```
>>> try:
        {some operation that may fail}
    except:
        {a method to handle error}
    else:
        {operation if the operation of `try`
         succeeds}
    finally:
        {operation that runs all the time}
```

In [8]:
```
# Let's assume you recruited participants with
# following identifiers:
# P01, P02, P03, ..., P30 (N = 30)
# Then for some reason you have missing participants
# (ex. P05, P18, P26 are missing).

# These are to simulate the scenario.
participant_ids = [f"P{str(i).zfill(2)}"
                   for i in range(1, 31)]
missing_ids = {"P05", "P18", "P26"}
# A data dictionary with missing participants
data = {pid: f"data_of_{pid}"
        for pid in participant_ids
        if pid not in missing_ids}

# Now, let's try to access data for all participants
# A for loop will iterate through all participant IDs
# so `pid` will take values from P01 to P30
for pid in participant_ids:
    print(f"Processing {pid}...")
    # This will raise KeyError for missing participants
    # because there's no key: 'P05'
    print(data[pid])
```

```
Processing P01...
data_of_P01
Processing P02...
data_of_P02
Processing P03...
data_of_P03
Processing P04...
data_of_P04
Processing P05...
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[8], line 23
     20 print(f"Processing {pid}...")
     21 # This will raise KeyError for missing participants
     22 # because there's no key: 'P05'
---> 23 print(data[pid])

KeyError: 'P05'
```

In [9]:
```python
# It's harder to know in advance
# which participant is missing
# if you have many more participants (ex. 100,000).
# This means that you may want to 'skip' missing data
# and continue processing other participants.

# This can be handled by using try/except
intact_participants = []
missing_participants = []
for pid in participant_ids:
    # So lines under `try` are the main lines
    # that you want to run
    try:
        # If `data[pid]` is not None or False,
        # `pid` is considered intact.
        if data[pid]:
            intact_participants.append(pid)
    # Assuming you don't know which error may happen,
    # you can use bare except to catch all exceptions.
    except:
        print(f"WARNING: Missing data for {pid}.")
        # You can further save these pids
        # into a list for later investigation.
        missing_participants.append(pid)
```

```
WARNING: Missing data for P05.
WARNING: Missing data for P18.
WARNING: Missing data for P26.
```

In [10]:
```python
# You can further investigate what to do with
# these participants.
print("Missing participants:", missing_participants)
```

```
Missing participants: ['P05', 'P18', 'P26']
```

Once you get more experienced, you would *anticipate* that `KeyError` is possible so write `except KeyError` to catch that specific error.

In [11]:
```python
for pid in participant_ids:
    try:
```

```
        if data[pid]:
            # pass for now...
            continue
    # Specifying which error you want to catch.
    # This means that other errors will
    # still raise exceptions.
    except KeyError:
        print(f"WARNING: Missing data for {pid}.")
```

```
WARNING: Missing data for P05.
WARNING: Missing data for P18.
WARNING: Missing data for P26.
```

There can be more than one type of error, and specifying only single error will not handle other error types. See the example below.

Let's suppose that we calculate some measure using the original values.

In [12]:
```
# This is what we mean:
data = {'P01': 2.6, 'P02': 1.5, 'P03': 2.0, 'P04': 3.1, 'P06': 4.0}
# The coder assumed that 'P01' - 'P06' exist without missing data,
# but 'P05' is missing.
participant_ids = [f'P{str(i).zfill(2)}' for i in range(1, 7)]

for pid in participant_ids:
    try:
        if data[pid]:
            # risky operation: x/(x-2)^2
            result = data[pid]/(data[pid]-2)**2
            print(f"{pid}: {result}")
    # When `pid` is 'P05', KeyError occurs
    except KeyError:
        print(f"WARNING: Missing data for {pid}.")
```

```
P01: 7.2222222222222205
P02: 6.0
```

```
---------------------------------------------------------------------
----
ZeroDivisionError                         Traceback (most recent call l
ast)
Cell In[12], line 11
      8 try:
      9     if data[pid]:
     10         # risky operation: x/(x-2)^2
---> 11         result = data[pid]/(data[pid]-2)**2
     12         print(f"{pid}: {result}")
     13 # When `pid` is 'P05', KeyError occurs

ZeroDivisionError: float division by zero
```

Huh, the loop breaks before it reaches the end of `data` with an error returned: `ZeroDivisionError`. This is because the operation of `try` fails when `pid`

is `'P03'`, because it's **2.0 / 0.0**, and there's no `except` specified for this error.

Here you can address the problem in two ways: 1) specify all possible errors in advance, or 2) throw a wide net using `exceptions`.

In [13]:
```
# 1) Catching multiple errors
for pid in participant_ids:
    try:
        if data[pid]:
            # risky operation: x/(x-2)^2
            result = data[pid]/(data[pid]-2)**2
            print(f"{pid}: {result}")
    except KeyError:
        print(f"WARNING: Missing data for {pid}.")
    except ZeroDivisionError:
        print(f"WARNING: Division by zero for {pid}.")
```

```
P01: 7.2222222222222205
P02: 6.0
WARNING: Division by zero for P03.
P04: 2.5619834710743796
WARNING: Missing data for P05.
P06: 1.0
```

In [14]:
```
# 2) Using a broad exception
for pid in participant_ids:
    try:
        if data[pid]:
            result = data[pid]/(data[pid]-2)**2
            print(f"{pid}: {result}")
    except Exception as e:
        print(f"WARNING: An error occurred for {pid}: {e}")
```

```
P01: 7.2222222222222205
P02: 6.0
WARNING: An error occurred for P03: float division by zero
P04: 2.5619834710743796
WARNING: An error occurred for P05: 'P05'
P06: 1.0
```

Without an error explicitly raised, we now see that `ZeroDivisionError` is raised for `'P03'`.

In [15]:
```
# Let's see the usecase of all four
# in a function.
def safe_divide(dict: dict, key: str) -> float | None:
    """
    Perform simple arithmetic division safely.

    Parameters
    ----------
    dict : dict
```

```
        A dictionary containing numeric values.
    key : str
        The key to access the value in the dictionary.

    Returns
    -------
    float | None
        The result of the division or None if an error occurred.
    """
    print(f"Currently processing key: {key}")
    try:
        result = dict[key] / (dict[key]-2)**2
    except Exception as e:
        print("Oops:", e)
        return None
    else:
        print(f"Division ended successfully: {result}")
        return result
    finally:
        # Blank line for better readability
        print()

for pid in participant_ids:
    safe_divide(data, pid)
```

```
Currently processing key: P01
Division ended successfully: 7.2222222222222205

Currently processing key: P02
Division ended successfully: 6.0

Currently processing key: P03
Oops: float division by zero

Currently processing key: P04
Division ended successfully: 2.5619834710743796

Currently processing key: P05
Oops: 'P05'

Currently processing key: P06
Division ended successfully: 1.0
```

4) Defensive programming & `raise`

Validate early; fail loudly and helpfully. You can **raise** your own exceptions.

In [16]:
```
# Use `raise` to trigger exceptions
value = -2.0
if value < 0:
```

```
    # Place your custom error message here
    # (ex. "Negative value error")
    raise ValueError("Negative value error")
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[16], line 6
      2 value = -2.0
      3 if value < 0:
      4     # Place your custom error message here
      5     # (ex. "Negative value error")
----> 6     raise ValueError("Negative value error")

ValueError: Negative value error
```

This again requires you to anticipate which error you will be raising.

In [17]:
```
def mean(values):
    if values is None:
        # If None is passed, raise ValueError
        raise ValueError("values cannot be None")
    if not hasattr(values, "__iter__"):
        # If values is not iterable, raise TypeError
        raise TypeError("values must be an iterable")
    values = list(values)
    if len(values) == 0:
        # If empty iterable is passed, raise ValueError
        raise ValueError("values must be non-empty")
    # If no error, compute the mean
    return sum(values) / len(values)

# This will work. `[1, 2, 3]` is not None,
# is iterable, and non-empty.
print(mean([1, 2, 3]))
```

```
2.0
```

In [18]:
```
# This will return the first ValueError
print(mean(None))
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[18], line 2
      1 # This will return the first ValueError
----> 2 print(mean(None))

Cell In[17], line 4, in mean(values)
      1 def mean(values):
      2     if values is None:
      3         # If None is passed, raise ValueError
----> 4         raise ValueError("values cannot be None")
      5     if not hasattr(values, "__iter__"):
      6         # If values is not iterable, raise TypeError
      7         raise TypeError("values must be an iterable")

ValueError: values cannot be None
```

In [19]:
```
# This will return the TypeError
print(mean(123))
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[19], line 2
      1 # This will return the TypeError
----> 2 print(mean(123))

Cell In[17], line 7, in mean(values)
      4     raise ValueError("values cannot be None")
      5 if not hasattr(values, "__iter__"):
      6     # If values is not iterable, raise TypeError
----> 7     raise TypeError("values must be an iterable")
      8 values = list(values)
      9 if len(values) == 0:
     10     # If empty iterable is passed, raise ValueError

TypeError: values must be an iterable
```

In [20]:
```
# This will return the second ValueError
print(mean([]))
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[20], line 2
      1 # This will return the second ValueError
----> 2 print(mean([]))

Cell In[17], line 11, in mean(values)
      8 values = list(values)
      9 if len(values) == 0:
     10     # If empty iterable is passed, raise ValueError
---> 11     raise ValueError("values must be non-empty")
     12 # If no error, compute the mean
     13 return sum(values) / len(values)

ValueError: values must be non-empty
```

5) Debugging in **Spyder** (IDE tips)

Breakpoints stop your program on a specific line so you can inspect variables.

Slides

Workflow:

1. Open your `.py` file in Spyder.
2. Click the **left gutter** next to a line number to set a red breakpoint dot. When you're debugging for the first time, set it next to the last line of your file.
3. Run with the **Debug** button (bug icon) or `Ctrl+F5`.
4. When execution stops:
   - Check **Variable Explorer** for current values.
   - Use **Debug current line (Ctrl+F10)** to run the next line.
   - Use **Step Into (Ctrl+F11)** to enter a function (downward arrow).
   - Use **Continue (Ctrl+F12)** to resume to the next breakpoint.
5. Fix the bug, re-run, repeat. Small steps win.

Pro tips:

- Keep functions small and pure; makes stepping and testing easier.

Practice: a bug you can squash

There's a subtle bug below. Use prints/logs or a debugger to find and fix it.

In [21]:
```
def normalize(values):
```

```
    # Normalize to 0..1 range
    vmin = min(values)
    vmax = max(values)
    # BUG: what if vmax == vmin ?
    return [(v - vmin) / (vmax - vmin) for v in values]

data = [3, 3, 3]
# See what's happening here. Do you expect a specific error?
normalize(data)
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[21], line 10
      8 data = [3, 3, 3]
      9 # See what's happening here. Do you expect a specific error?
---> 10 normalize(data)

Cell In[21], line 6, in normalize(values)
      4 vmax = max(values)
      5 # BUG: what if vmax == vmin ?
----> 6 return [(v - vmin) / (vmax - vmin) for v in values]

ZeroDivisionError: division by zero
```

> **Your task:** handle the edge case where all values are identical.
> *Hint:* return a list of zeros in that case, or choose a behavior and document it.

In [22]:
```
# ✅ One possible fix:
def normalize_fixed(values):
    vmin = min(values)
    vmax = max(values)
    if vmax == vmin:
        return [0.0 for _ in values]  # documented behavior
    return [(v - vmin) / (vmax - vmin) for v in values]

print("`values`: [3, 3, 3]; "
      f"function return: {normalize_fixed([3, 3, 3])}")
print()
print("`values`: [1, 2, 3]; "
      f"function return: {normalize_fixed([1, 2, 3])}")
```

```
`values`: [3, 3, 3]; function return: [0.0, 0.0, 0.0]

`values`: [1, 2, 3]; function return: [0.0, 0.5, 1.0]
```

6) `assert` and quick checks

`assert <condition>, "message"`

- `assert` tests whether a `<condition>` is `True`.
- `AssertionError` is raised with `"message"` if `<condition>` is `False`.
- Great for catching impossible states during development.

Developers use `assert` mainly because:

- It's **concise** and **idiomatic** for sanity checks.
- It clearly signals assumptions.

In [23]:
```
def compute_ratio(a, b):
    # if b is zero,
    # raise an AssertionError with message:
    # "b must be non-zero"
    assert b != 0, "b must be non-zero"
    return a / b

# triggers an AssertionError with our message
compute_ratio(10, 0)
```

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[23], line 9
      6     return a / b
      8 # triggers an AssertionError with our message
----> 9 compute_ratio(10, 0)

Cell In[23], line 5, in compute_ratio(a, b)
      1 def compute_ratio(a, b):
      2     # if b is zero,
      3     # raise an AssertionError with message:
      4     # "b must be non-zero"
----> 5     assert b != 0, "b must be non-zero"
      6     return a / b

AssertionError: b must be non-zero
```

In [24]:
```
# Of course, it can be replaced with if-statement.
def compute_ration(a, b):
    if b == 0:
        raise ValueError("b must be non-zero")
    return a / b

# Now ValueError is raised,
# but the message is the same.
compute_ration(10, 0)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[24], line 9
      5     return a / b
      7 # Now ValueError is raised,
      8 # but the message is the same.
----> 9 compute_ration(10, 0)

Cell In[24], line 4, in compute_ration(a, b)
      2 def compute_ration(a, b):
      3     if b == 0:
----> 4         raise ValueError("b must be non-zero")
      5     return a / b

ValueError: b must be non-zero
```

Wrap-up

You can't avoid errors. You **can** learn to make them boring:

- Read the traceback bottom-up
- Catch specific exceptions
- Raise clear, helpful errors
- Use Spyder's debugger + Variable Explorer to look *inside* your code
- Fix in small, verified steps

You've got this. Future-you debugs in half the time. 🚀